

Lab 3 — Batch B

Scheme (DrRacket, #lang sicp) · 15:30–16:55

Language: Scheme (DrRacket, `#lang sicp`)

Assume the following are already available to you exactly as defined below — copy this block into your file. `map` is already built into your language, so just use it directly; use *these* definitions of `length` / `filter` / `foldr` / `foldl` rather than reaching for any other version.

```
(define (length lst)
  (define (length-iter n lst) (if (null? lst) n (length-iter (+ 1 n) (cdr lst))))
  (length-iter 0 lst))

(define (filter pred l)
  (if (null? l) nil
      (let ((x (car l)))
        (if (pred x) (cons x (filter pred (cdr l))) (filter pred (cdr l))))))

(define (foldr f v l)
  (if (null? l) v (f (car l) (foldr f v (cdr l)))))

(define (foldl f v l)
  (if (null? l) v (foldl f (f v (car l)) (cdr l))))
```

Q1 (1 mark) — `average-hot-temperature` (file name: `q1.scm`)

Write a procedure `average-hot-temperature` that takes a list of Celsius temperatures and a Fahrenheit threshold, and returns the average Fahrenheit temperature of just the days that come out strictly above that threshold ($F = C * 9/5 + 32$). Assume **at least one** day comes out above the threshold, so you don't need to handle dividing by zero. Use `map` together with `filter`, `foldr`, and `length` — don't write a hand-rolled recursive loop.

```
(average-hot-temperature '(0 20 30 100 25) 70) => 125
```

Q2 (1 mark) — `count-hot-days` (file name: `q2.scm`)

Write a procedure `count-hot-days` that takes a list of Celsius temperatures and a Fahrenheit threshold, and returns how many of the (converted) temperatures are strictly above that threshold. Use `map` together with `filter` and `length` — don't write a hand-rolled recursive loop.

```
(count-hot-days '(0 20 30 100 25) 70) => 3
(count-hot-days '(0 5 10) 70)           => 0
```

Q3 (1 mark) — hot-days-reversed (file name: q3.scm)

Write a procedure `hot-days-reversed` that takes a list of Celsius temperatures and a Fahrenheit threshold, and: converts them all to Fahrenheit, keeps only the ones strictly above the threshold, and returns the result **in reverse of the order they'd naturally come out** — using `map`, `filter`, and `foldl` (specifically `foldl`, not `foldr`, and not a built-in `reverse`). Think about how `foldl`'s accumulator gets threaded through, and what happens if you `cons` each element onto it as you go.

```
(hot-days-reversed '(0 20 30 100 25) 70) => (77 212 86)
```

(Sanity check: `map` + `filter` alone, without the fold, would give you `(86 212 77)` — in forward order. Your job is to also flip that order, using nothing but `foldl`.)

Take-home practice (ungraded)

total-warming-needed

Write `total-warming-needed`, which takes a list of Celsius temperatures and a Fahrenheit threshold, and returns the **total number of degrees** needed across all the days that are *below* that threshold to bring each one up to it (e.g. a day at 32°F with a threshold of 70°F needs 38 more degrees). Use `map`, `filter`, and `foldr`.

```
(total-warming-needed '(0 20 30 100 25) 70) => 40 ; 38 + 2
```

Once you're done, feel free to also try Batch A's Q1–Q3 for extra practice.